

PyPTO-Gym 仓正式开源

算子随心练，模型轻松连

高质量融合算子样例：从学习实践到真实大模型快速集成

80

融合算子

10

模型适配

100%

基于 CANNBot 开发

1.12x

平均模型加速



OVERVIEW · 概述

PyPTO Gym仓（算子训练场）正式开源

PyPTO 算子样例仓库

- 基于 PyPTO 高效编程框架，提供**融合算子开发样例**与**大模型适配样例**
- 基于 **CANNBot** 完成本仓算子设计、开发与调优

相关开源仓



目标用户

- **算法开发者**: 快速复用或参考开发新算子，验证算法，专注于算法逻辑
- **应用开发者**: 高效迁移和集成部署大模型，落地业务，深耕于场景需求

把专家路径拆成可学习、可验证、可迁移的样例



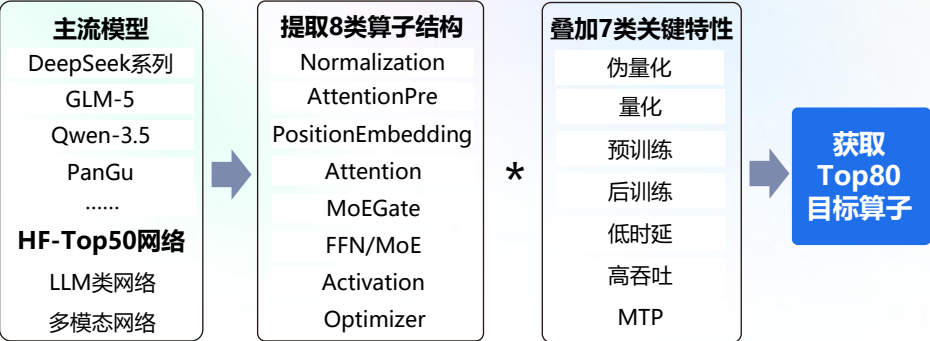
OVERVIEW · 概述

使用 CANNBot 完成Top80算子构建与10+模型适配

基于Top模型 筛选Top80目标算子

整网自动适配

贡献GYM仓



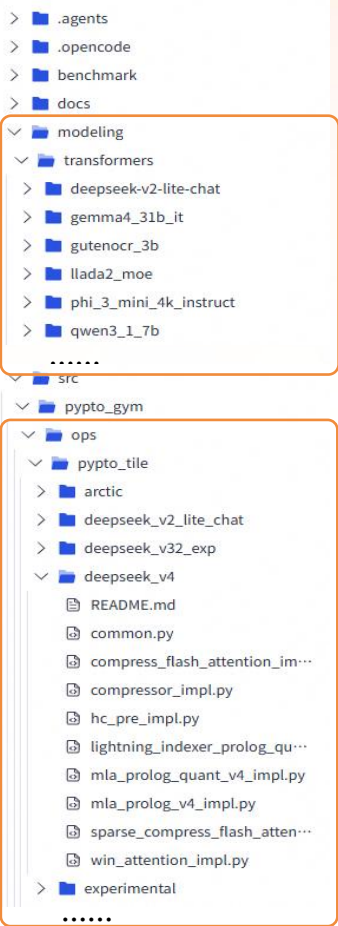
top模型	模型结构特点
Qwen3.6-27B	GDR/GQA混合
Deepseek-V2-Lite-Chat	Prolog + MLA
Qwen3-1.7B	GQA + Dense + RMSNorm
Spatial-SSRL-3b	GQA + Dense + Rope
Gemma4-31B	GQA + Dense + FFN
LLaDa 2.1-mini	MHA + MoE + Gate
Phi-3-mini-128k-instruct	GQA + MoE + RMSNorm
.....	

- 已完成10个模型替换验证，替换主要结构后5~20%性能提升；
- 完成整网适配skills开发，具备单卡模型agent入网适配能力。

Top80算子自动生成

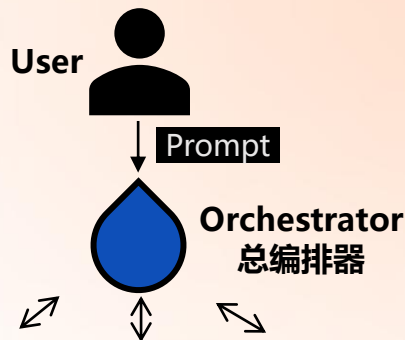
用例分级	典型算子	Top算子数量
level4	sparse_compress_flash_attention、glm_attention_fusion、GQA、GDR	23
level3	mha_prolog_quant_v4、glm_moe_fusion、glm_ffn_shared_expert_quant	50
level1~2	softmax、glm_gate、group_matmul	9
总计	-	82

- 基于简单交互生成中等与典型FA复杂融合算子；
- 基于多轮交互生成FAG、新增FA变体等复杂算子；



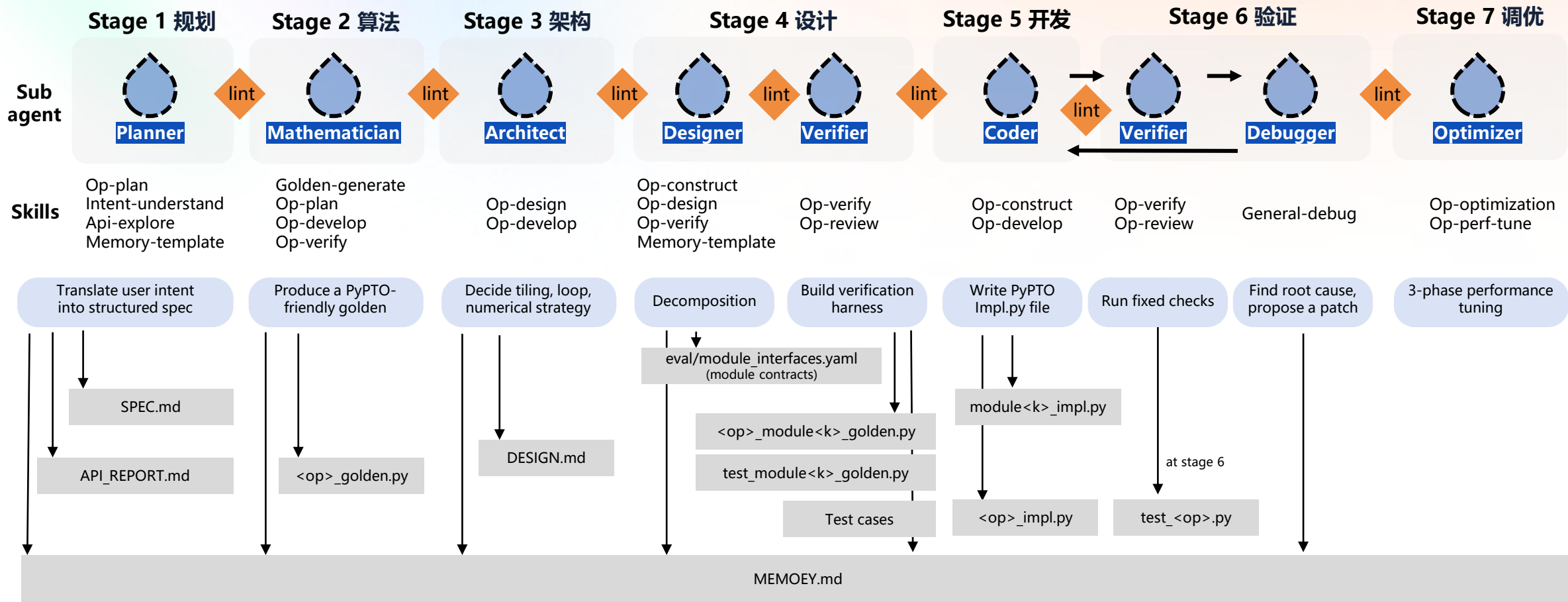
OPERATOR · 算子开发

CANNBot 自动开发流程



从「手写 kernel」到「描述一个算子」
算子生成从未如此简单

9智能体团队 / Stage1-7自动开发 / 逐阶段lint门禁守护



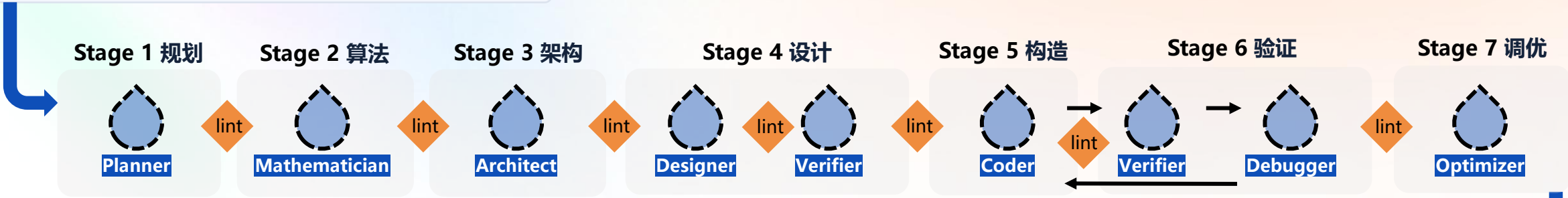
算子公式

```
q, k = l2norm(q2d) / sqrt(D), l2norm(k2d) .....:1
g_cum, decay = tril @ gate_v, exp((g_cum[:, None] - g_cum[None, :]) * tril) .....:2
k_beta, A_inv = k * beta, inverse_lower_triangular(I - ((k * beta) @ k.T) * decay * mask) .....:3
value_out, v_prime = A_inv @ (v * beta), (A_inv @ (k_beta * exp(g_cum))) @ state.T .....:4
attn_tmp, qg, kg = (q @ k.T) * decay * tril, q * exp(g_cum), k * exp(g_cum[-1] - g_cum) .....:5
chunk_out = qg @ state.T + attn_tmp @ (value_out - v_prime) .....:6
cur_state = state * exp(g_cum[-1]) + (value_out - v_prime).T @ kg .....:7
```

OPERATOR · 代表算子

算子实例：Gated Delta Rule

Qwen3.5-9B 的线性注意力融合样例



工程价值

- **复杂kernel 融合**：GDR中包含L2Norm/decay/求逆/有前后依赖的状态更新等多个复杂计算模块，包含多个矩阵乘计算，中间穿插vector计算。传统ascendc代码实现难度大，PyPTO最终生成的kernel代码行数在300行以内，同时相比小算子可以取得8.28x性能收益。
- **快速实现部署验证**：PyPTO算子接口与PyTorch十分亲和，基于python的代码实现逻辑与tensor编程框架使得算子能够快速进行验证，并在网络中实现算子替换

性能验证 Atlas A2推理服务器 · FP32 · 用例提速

算子规格	小算子耗时	PyPTO融合算子耗时	加速比
B1 T256 H32 D128 1-die seq256 prefill	14116us	1704μs	8.28x

算子代码

```
@pypto.frontend.jit(runtime_options={...})
def _gdr_kernel(...):
    for b_idx in pypto.loop(b, name="LOOP_B_TND", idx_name="b_idx"):
        for nv_idx in pypto.loop(nv, name="LOOP_Nv_TND", idx_name="nv_idx"):
            for s_idx in pypto.loop(s, l, name="LOOP_S_TND", idx_name="s_idx",
                                   unroll_list=[16, 1]):
                # l2norm + scale
                q = q2d / pypto.sqrt((q2d * q2d).sum(-1, keepdim=True) + eps)
                k = k2d / pypto.sqrt((k2d * k2d).sum(-1, keepdim=True) + eps)
                q = q * (1 / d ** 0.5)

                # pre_attn
                gate_cum = pypto.matmul(tril, gate_v, pypto.DT_FP32)
                decay = ((gate_cum - gate_cum.transpose(0, 1)) * tril).exp()

                k_beta = k * beta_v
                a = pypto.matmul(k_beta, k, pypto.DT_FP32, b_trans=True) * decay * mask
                # inverse(A) 核心
                a21_inv = pypto.matmul(pypto.matmul(i2, a21, pypto.DT_FP32), i1,
                                       pypto.DT_FP32)

                # value_out & k_cumdecay
                value_out = pypto.matmul(a_inv, v2d * beta_v, pypto.DT_FP32)
                v_prime = pypto.matmul(k_cum, state, pypto.DT_FP32, b_trans=True)

                # recurrent state attention
                k_cum = pypto.matmul(a_inv, k_beta * pypto.exp(gate_cum), pypto.DT_FP32)
                kg = k * (gate_cum[1-1:1, :] - gate_cum).exp()
                qg = q * gate_cum.exp()
                attn_tmp = attn * decay * tril
                attn_inter = pypto.matmul(qg, state, pypto.DT_FP32, b_trans=True)
                attn = pypto.matmul(q, k, pypto.DT_FP32, b_trans=True)

                cur_state = state * gate_cum.exp()[1-1:1, :] + pypto.matmul(value_out, kg,
                                                                           a_trans=True) - pypto.matmul(v_prime, kg, a_trans=True)
                chunk_out = attn_inter + pypto.matmul(attn_tmp, value_out) - pypto.matmul(attn_tmp, v_prime)
```

INTEGRATION · 模型集成

模型迁移与算子集成流程

基于 `pypto-fused-op-integration` skill 完成零代码自动化迁移与替换集成



NPU 迁移与基线

阶段 1

- 确认 HF/权重/tokenizer
- 生成 ask · bench · prof
- baseline 跑通后建 Git 基线



融合推荐

阶段 2

- 读模型源码与算子模式
- 匹配 Attention/MoE/FFN 案例
- 确认目标替换边界



理解验证

阶段 3

- 打点采集真实 shape/dtype
- 生成 test_cases.json
- 编写 golden 并验证语义



算子生成

阶段 4

- Benchmark 自动化或手动开发
- planner→coder→verifier→optimizer
- 产出 impl · golden · test



替换集成

阶段 5

- 创建 pto_kernels 适配层
- USE_PTO_* 按算子开关
- sys.modules 注入 + fallback



端到端验证

阶段 6

- ask 双模式 (eager/aclgraph) 运行
- bench/prof 采集性能与 trace
- 归档提交, 贡献社区 (可选)

PRACTICE · 模型实践

模型实践：Qwen3.5-9B迁移与集成

算子 kernel 如何接入整网：整网迁移到torch_npu→ 开发pypto算子 → 通过sys.modules注入将算子替换进网络 → 端到端精度+性能验证

适配接入

```
# 1. ops/.../qwen3_5_9b/__init__.py 打包适配层
from .gated_delta_rule.gated_delta_rule_impl \
    import gated_delta_rule_wrapper # 导入PyPTO实现

# 2. ask_Qwen3.5-9B.py: 注入模型, 须在import transformers 之前
if args.use_pypto:
    sys.path.insert(0, model_path)
    import qwen3_5_9b_pto_kernels as pto
    sys.modules["qwen3_5_9b_pto_kernels"] = pto # 注入sys.modules
    pto.USE_PTO_GATED_DELTA_RULE = True

# 3. forward阶段接管对应module执行前向计算
for m in model.modules():
    if type(m).__name__=="Qwen3_5GatedDeltaNet":
        def _patched(q,k,v,**kw): # monkey-patch接管
            try: return pto.gated_delta_rule_wrapper(
                q,k,v,**kw)
            except NotImplementedError:
                return orig_chunk(q,k,v,**kw)
        m.chunk_gated_delta_rule = _patched
```

接入关键点



Agent赋能加速模型适配

提供pypto-fused-op-integration skill, agent自行完成包含适配torch_npu接口, 算子替换以及回归验证等大部分网络迁移过程。



Agent赋能高效算子开发

识别出优化场景, 在pypto仓下面进行算子开发。开发过程可实现agent全程自主管理, 用户零介入。



零侵入算子注入

通过sys.modules方式注入pypto, 并将GDR原地替换, 签名一致; 以及内嵌 modeling / 运行时 monkey-patch 两种方式

2.24×

整网推理加速

1419→3178 TPS

`gated_delta_rule` 融为 1 kernel

B1 T256 H32 D128 1-die seq256 prefill

GET STARTED · 快速上手

快速上手



1 部署 PyPTO 环境

按官方文档部署并编译安装 PyPTO
或基于PyPTO CANNLab部署运行



2 运行算子测试

`pytest tests/ops/<model>`



3 端到端推理

`python modeling/infer.py`

基于 CANNBot 能力将流程自动化

自动化路径	提示词	核心流程	产出
 Agent 开发算子	“开发 softmax 算子, fp16, shape [1,128]/[4,2048]/[32,4096]”	需求 · API · Golden · 设计 · 编码 · 精度 · 性能	算子目录 custom/softmax/: impl · golden · test
 Skill 模型接入	“把 Qwen3.5-9B 迁到昇腾 NPU, 集成 PyPTO 融合算子”	NPU 迁移 · 基线 · Golden · 开发 · 集成 · 验证 · 归档	集成 modeling + pto_kernels 算子库 + ask/bench 脚本, NPU 端到端跑通并归档

学习 · 实践 · 迁移 · 共建

THANKS



Star & Fork

gitcode.com/cann/pypto-gym



Issue · PR

贡献算子或修复问题



社区讨论

CANN 开发者社区

